

Adult Income Prediction — Day 4 Write-Up

Setup and pipeline

I reused the Day 1 hold-out split (`random_state=42`) and the Day 3 engineered features inside a single sklearn `Pipeline` (feature engineering → preprocessing → model). All estimators use a fixed `random_state` . Library versions are printed in the notebook.

I dropped `fnlwgt` for Day 4. It is a census sampling weight, not a direct income signal, and it was not important in Day 3. I kept both `has_capital_gain` and `log_capital_gain` . They overlap, so logistic regression can give the binary flag a large negative coefficient once the log amount is in the model. That is redundancy, not a claim that capital gains lower income.

Hyperparameter search

Searches optimized **F1** with stratified 3-fold CV (`n_jobs=-1`).

Model	Search	Best CV F1	Best settings (short)
Logistic Regression	Grid over penalty, C, class_weight	0.6898	L1, C=1.0, class_weight=balanced
Random Forest	RandomizedSearch n_iter=30	0.6829	n_estimators=100, max_depth=16, max_features=0.3, min_samples_leaf=1
HistGradientBoosting	RandomizedSearch n_iter=40	0.7094	learning_rate=0.15, max_iter=100, max_depth=None, l2=0.5, min_samples_leaf=10

Winner for the final pipeline: HistGradientBoosting.

Compared with Day 3 untuned HGB (~0.708 CV F1), tuning gave a small lift. Logistic regression benefited clearly from `class_weight='balanced'` (Day 2 follow-up).

Diagnostics

- **Learning curve (HGB):** train and CV F1 move together as more training data is added. The remaining gap is moderate — not the extreme overfitting we saw with an unconstrained Day 2 tree.
- **C curve (LR):** very small C underfits; large C can overfit. $C \approx 1$ worked well with balanced class weights.
- **max_depth (HGB):** shallow depths underfit; leaving depth unlimited with `l2_regularization=0.5` and `min_samples_leaf=10` was preferred by search.

Concrete fixes used: keep leaf / L2 regularization on HGB; use balanced weights on LR; do not go back to an unconstrained deep tree.

Calibration and threshold

I applied isotonic calibration (`CalibratedClassifierCV`) and chose the decision threshold on a validation slice of the **training** data to maximize F1.

- Best validation threshold: **0.33** (val F1 $\approx$ 0.73)
- Default 0.50 is precision-heavy on this imbalanced problem; 0.33 trades some precision for higher recall and better F1

Hold-out test results (used once)

Setup	Accuracy	Precision	Recall	F1	ROC AUC	Brier
Uncalibrated @ 0.50	0.8748	0.7843	0.6578	0.7155	0.9298	0.0868
Calibrated @ 0.50	0.8768	0.7972	0.6506	0.7164	0.9299	0.0867
Calibrated @ 0.33	0.8571	0.6707	0.7917	0.7262	0.9299	0.0867

Tuned threshold test confusion matrix: FP=909, FN=487 (fewer missed high earners than at 0.50).

Day 2 logistic regression hold-out F1 was 0.6630; Day 3 untuned HGB was around 0.72 on a reference hold-out. The final calibrated + thresholded HGB reaches **F1 = 0.7262** on the sealed test set.

Artifact and production note

Saved: `final_income_pipeline.joblib` (calibrated pipeline + threshold + metadata).

```
import joblib
artifact = joblib.load("final_income_pipeline.joblib")
proba = artifact["model"].predict_proba(X_new)[: , 1]
pred = (proba >= artifact["threshold"]).astype(int)
```

If the live population's class balance changes, recalibrate or retune the threshold. Retrain with the same `RANDOM_STATE` and pipeline code to reproduce this run.